

在算法竞赛中，求组合数 $\binom{n}{m}$ （即从 n 个元素中选 m 个的方案数）是非常基础且高频的考点。因为组合数的增长速度极快，竞赛中通常要求对一个大质数（如 $10^9 + 7$ 或 998244353）取模。

根据数据范围的不同，竞赛中最常用的组合数模板有以下三种。

1. 阶乘与乘法逆元法 (最常用，适用于 $N \leq 10^6$)

这是竞赛中 **最标准、使用最广** 的模板。利用公式 $\binom{n}{m} = \frac{n!}{m!(n-m)!} \pmod{P}$ 。

除法取模需要用到**费马小定理**求乘法逆元。我们在预处理时，先求出所有阶乘，再用快速幂求出最大阶乘的逆元，最后倒推求出所有逆元。

- **时间复杂度**：预处理 $O(N)$ ，单次查询 $O(1)$ 。

C++

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 1e6 + 5;    // 根据题目修改最大范围
5  const int MOD = 1e9 + 7;    // 常见的模数，必须是质数
6
7  long long fac[MAXN]; // 阶乘数组
8  long long inv[MAXN]; // 阶乘逆元数组
9
10 // 快速幂计算 a^b % MOD
11 long long qpow(long long a, long long b) {
12     long long res = 1;
13     a %= MOD;
14     while (b > 0) {
15         if (b & 1) res = res * a % MOD;
16         a = a * a % MOD;
17         b >>= 1;
18     }
19     return res;
20 }
21
22 // 预处理阶乘和逆元
23 void init() {
24     fac[0] = inv[0] = 1;
25     // 1. 预处理阶乘
26     for (int i = 1; i < MAXN; i++) {
27         fac[i] = fac[i - 1] * i % MOD;
28     }
29     // 2. 费马小定理求最大阶乘的逆元
30     inv[MAXN - 1] = qpow(fac[MAXN - 1], MOD - 2);
31     // 3. 线性倒推求所有逆元: 1/i! = (i+1)/(i+1)!
32     for (int i = MAXN - 2; i >= 1; i--) {
33         inv[i] = inv[i + 1] * (i + 1) % MOD;
34     }
35 }
36
37 // 求组合数 C(n, m)
```

```

38 long long C(int n, int m) {
39     if (m < 0 || m > n) return 0; // 不合法的情况
40     // C(n, m) = n! * (m!)^-1 * ((n-m)!)^-1 % MOD
41     return fac[n] * inv[m] % MOD * inv[n - m] % MOD;
42 }
43
44 int main() {
45     init(); // 必须先调用初始化函数
46     // cout << C(5, 2) << endl; // 输出 10
47     return 0;
48 }

```

2. 递推法 / 杨辉三角 (适用于 $N \leq 2000$)

利用组合数递推公式 $\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$ 。不需要用到求逆元，适合模数不是质数，或者数据范围较小的情况。

- **时间复杂度**：预处理 $O(N^2)$ ，单次查询 $O(1)$ 。
- **空间复杂度**： $O(N^2)$ ，因此 N 最大只能开到 2000 到 5000 左右。

C++

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 2005;
5  const int MOD = 1e9 + 7; // 如果题目不要求取模，去掉模数运算即可（但要注意爆 long long）
6  long long c[MAXN][MAXN];
7
8  void init() {
9      for (int i = 0; i < MAXN; i++) {
10         c[i][0] = 1; // C(i, 0) = 1
11         for (int j = 1; j <= i; j++) {
12             c[i][j] = (c[i - 1][j - 1] + c[i - 1][j]) % MOD;
13         }
14     }
15 }
16
17 long long C(int n, int m) {
18     if (m < 0 || m > n) return 0;
19     return c[n][m];
20 }

```

3. Lucas 定理 (适用于 N, M 极大，但模数 P 较小)

当 N, M 达到 10^{18} 级别，完全无法预处理，但模数 P 比较小（通常 $P \leq 10^5$ 且必须是质数）时使用。

核心公式： $\binom{n}{m} \equiv \binom{n \bmod P}{m \bmod P} \times \binom{n/P}{m/P} \pmod{P}$ 。

- **时间复杂度**： $O(P)$ 预处理（如果多次查询），单次查询 $O(P \log_P N)$ 。

C++

```

1  #include <bits/stdc++.h>

```

```

2  using namespace std;
3
4  long long MOD; // 注意: Lucas定理要求模数必须是较小的质数
5
6  // 快速幂
7  long long qpow(long long a, long long b) {
8      long long res = 1;
9      while (b) {
10         if (b & 1) res = res * a % MOD;
11         a = a * a % MOD;
12         b >>= 1;
13     }
14     return res;
15 }
16
17 // 在模 P 较小的情况下, 直接 O(P) 暴力算小组合数
18 long long C(long long n, long long m) {
19     if (m < 0 || m > n) return 0;
20     if (m > n - m) m = n - m;
21     long long up = 1, down = 1;
22     for (long long i = 1; i <= m; i++) {
23         up = up * (n - i + 1) % MOD;
24         down = down * i % MOD;
25     }
26     return up * qpow(down, MOD - 2) % MOD;
27 }
28
29 // 递归应用 Lucas 定理
30 long long Lucas(long long n, long long m) {
31     if (m == 0) return 1;
32     return C(n % MOD, m % MOD) * Lucas(n / MOD, m / MOD) % MOD;
33 }
34
35 int main() {
36     MOD = 10007; // 假设较小质数模数
37     // cout << Lucas(10000000000LL, 5000000000LL) << endl;
38     return 0;
39 }

```

在数论和算法竞赛中, 最大公约数 (GCD)、最小公倍数 (LCM) 以及同余理论是极为重要的基础。特别是在处理大数取模时, 稍有不慎就会溢出或得出负数。

以下是为你整理的实用模板和竞赛避坑指南:

1. GCD 与 LCM (最大公约数与最小公倍数)

虽然 C++17 在 `<numeric>` 库中直接提供了 `std::gcd` 和 `std::lcm`, 但在很多环境下 (或者为了拓展到扩展欧几里得) 我们依然习惯手写。

竞赛避坑警告: 求 LCM 时, **一定要先除后乘** `a / gcd(a,b) * b`, 否则 `a * b` 极易在第一步就爆 `long long`。

C++

```

1  #include <iostream>
2  using namespace std;
3
4  // 欧几里得算法（辗转相除法）
5  long long gcd(long long a, long long b) {
6      return b == 0 ? a : gcd(b, a % b);
7  }
8
9  // 最小公倍数（先除后乘防溢出）
10 long long lcm(long long a, long long b) {
11     if (a == 0 || b == 0) return 0;
12     return a / gcd(a, b) * b;
13 }

```

2. 同余基本运算模板

在竞赛中，所有的加减乘法都要随时取模。**最大天坑**：C++ 中的 `%` 运算符对于负数会返回负数（例如 `-2 % 5 = -2`），而在数学同余中我们需要正余数（应为 `3`）。因此减法取模必须特殊处理。

C++

```

1  long long MOD = 1e9 + 7;
2
3  // 加法同余
4  long long add(long long a, long long b) {
5      return (a % MOD + b % MOD) % MOD;
6  }
7
8  // 乘法同余
9  long long mul(long long a, long long b) {
10     return (a % MOD * (b % MOD)) % MOD;
11 }
12
13 // 减法同余（必须加 MOD 再取模，防止 C++ 负数取模坑）
14 long long sub(long long a, long long b) {
15     return ((a % MOD - b % MOD) + MOD) % MOD;
16 }

```